- **Comente sobre tratamento de exceção:** O tratamento de exceções (geralmente implementado com blocos `try...catch` ou middlewares de erro no Express) serve para capturar falhas e erros inesperados durante a execução do código, impedindo que o servidor vá abaixo (*crash*) e permitindo retornar uma resposta HTTP adequada e controlada ao cliente.

- **O que é uma requisição síncrona?** É uma operação executada de forma sequencial e bloqueante, em que cada linha de código ou instrução tem de aguardar obrigatoriamente que a anterior termine por completo para que a execução possa avançar, bloqueando a *thread* principal.

- **O que é uma requisição assíncrona?** É uma operação não bloqueante que permite ao programa continuar a processar outras tarefas e atender a novos pedidos enquanto aguarda a conclusão de tarefas demoradas, como consultas a bases de dados ou pedidos a APIs externas.

- **Para que serve o comando await?** O comando `await` é utilizado dentro de funções assíncronas (`async`) para pausar a execução do código até que a `Promise` associada seja resolvida ou rejeitada, permitindo lidar com operações assíncronas de forma legível e sequencial.

- **Para que serve o método findOne({email})?** É um método utilizado em ORMs/ODMs (como o Mongoose no MongoDB ou Sequelize no SQL) para procurar e retornar o **primeiro** registo na base de dados cujo campo `email` corresponda ao valor fornecido no objeto de filtro.

- **Explique o código: `bcrypt.compare(password, user.password)`:** Esta linha utiliza a biblioteca `bcrypt` para comparar de forma segura uma palavra-passe em texto simples introduzida pelo utilizador (`password`) com o hash encriptado correspondente guardado na base de dados (`user.password`), retornando `true` se coincidirem ou `false` caso contrário.